

Potential-based constraints for LLM-guided reward design in financial reinforcement learning

Abstract

Large language model (LLM)-guided reinforcement learning can inject financial knowledge through generated state and reward functions, but adding an unconstrained generated scalar directly to the task reward can distort the investment objective. We propose PBIR-GIFT, a potential-based intrinsic reward extension of the GIFT state-reward interface framework. PBIR-GIFT interprets the generated scalar as a bounded market-state potential and converts changes between consecutive potentials into reward shaping. The existing callable interface is retained, keeping language-model generation offline and requiring no online LLM calls. We evaluate the method on two five-equity subsets derived from the FINS-ABER benchmark, six rolling half-year windows, and three random seeds using a fixed-code paired protocol. PBIR-GIFT improved mean cell-level Sharpe by 0.278 over Pure GIFT in 10 of 12 dataset-window cells (Hedges' $g = 0.940$), although the hierarchical-bootstrap 95% confidence interval $[-0.010, 0.558]$ marginally crossed zero. The evidence therefore supports a favorable but conditional effect rather than universal superiority, while demonstrating a low-intrusion way to constrain LLM-generated reward interfaces in financial reinforcement learning.

Keywords: potential-based reward shaping, large language models, financial reinforcement learning, portfolio optimization, reward hacking, generalization

1 Introduction

Background. Financial portfolio trading requires sequential rebalancing under changing returns, risks, and transaction costs, making deep reinforcement learning (DRL) a natural choice [11, 14, 20, 29, 30, 36, 38]. Prior studies improve policy architectures and optimization [19, 23, 26], enrich market states with technical, relational, positional, or memory information [9, 17, 42], or redesign rewards using smoothing, risk penalties, imitation, and forward-looking objectives [9, 10, 15, 18]. More recently, large language models (LLMs) have served as financial agents, signal generators, policy components, and reward designers [4, 8, 35, 39]. Together, these methods expand financial RL but leave its state-reward interface insufficiently constrained.

Limitations. Three common limitations motivate this study. **(1) Accuracy.** OHLCV observations leave relevant financial relationships implicit, while short-horizon returns provide noisy supervision for long-term risk-adjusted performance

[18, 42]. **(2) Efficiency.** Repeated rollouts, multi-agent interaction, and online LLM inference increase training and deployment costs [8, 35, 39]. **(3) Generalization.** Hand-designed interfaces may not transfer across assets or regimes, whereas unrestricted LLM rewards may encode spurious patterns, inconsistent reasoning, or weak risk control [7, 16, 40, 41]. These limitations call for better reward guidance without another decision layer or LLM inference during portfolio execution.

Contributions. We propose PBIR-GIFT, an environment-side extension of GIFT [33]. **(1) Accuracy.** PBIR reinterprets the LLM-generated scalar as a bounded measure of market-state desirability and rewards improvement between consecutive states instead of adding the scalar directly to the investment reward. **(2) Efficiency.** Code generation remains offline, the existing callable interface is retained, and portfolio execution requires no online LLM calls. **(3) Generalization.** The same asset-independent reward transformation is applied across Dataset 1, Dataset 2, and six rolling windows. Mean Sharpe increased by 0.278 in 10 of 12 fixed-code cells, with positive dataset-level differences of +0.220 and +0.336, although the 95% interval marginally crossed zero. Under its stated assumptions, the transformation preserves the optimal policies of the reference reward, but it does not imply identical finite-sample PPO solutions [21].

2 Related Work

2.1 Deep Reinforcement Learning and Interface Design

Portfolio DRL maps market and holding states to rebalancing actions and learns from return- and risk-related feedback. Research has progressed from direct policy optimization [20] to deep multi-asset and relation-aware models [14, 30, 36], standard actor-critic algorithms [11, 19, 23, 26], enriched states [9, 17, 42], and smoothed, risk-sensitive, imitative, or forward-looking rewards [9, 10, 15, 18].

The common limitation is that most methods optimize the policy while treating the state-reward interface as a fixed design choice. Such interfaces can omit latent financial relations, produce noisy feedback, and require manual redesign when assets or regimes change.

2.2 LLM-Guided Financial Decision-Making

Financial LLMs support domain understanding, sentiment analysis, reasoning, and benchmarking [22, 32, 34, 37]. Trading systems further use them as agents, signal generators, RL components, and risk-sensitive advisers [4, 8, 35, 39]. GIFT instead assigns the LLM an offline role: it generates a state-reward interface while PPO remains the portfolio decision maker [33].

The common limitation is that an unconstrained generated reward can introduce unsupported patterns, inconsistent actions, look-ahead bias, or unstable out-of-sample behaviour [7, 40]. In GIFT, the generated intrinsic reward is still added directly to the trading reward, leaving its influence without a policy-invariant shaping constraint.

2.3 Potential-Based Reward Shaping

Potential-based reward shaping augments a reference reward with $F(s_t, s_{t+1}) = \gamma\Phi(s_{t+1}) - \Phi(s_t)$. Under standard discounted Markov decision-process conditions, this transformation preserves the ordering of optimal policies for the reference objective [21].

Its limitation in the present context is not the shaping theorem itself, but the absence of an established mechanism for constraining an LLM-generated financial reward. PBIR-GIFT connects these lines by treating the generated scalar as $\Phi(s)$ and letting the environment construct the transition-level shaping term.

3 Method

PBIR-GIFT changes only how GIFT’s generated scalar enters PPO. The existing portfolio environment constructs states and executes allocations; GIFT generates and validates two callables offline; PBIR interprets the scalar output as a bounded potential and rewards its change between consecutive states. The frozen interface is then reused during PPO training and rolling-window evaluation (Fig. 1).

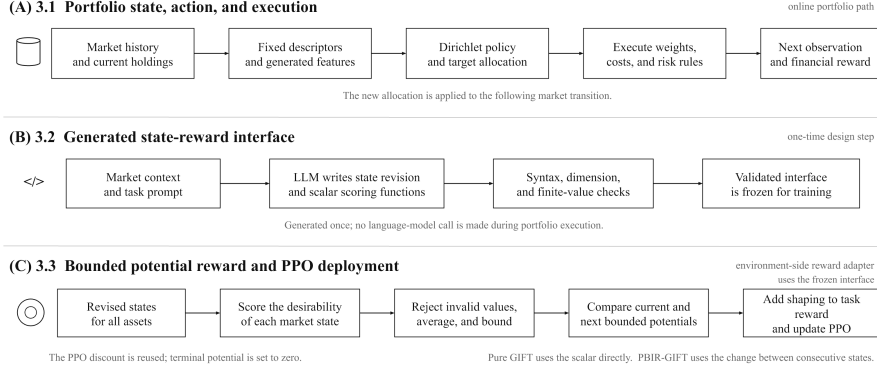


Fig. 1 PBIR-GIFT method. Panels a-c follow Sections 3.1-3.3. Panel a shows portfolio state construction and execution, panel b shows one-time interface generation and validation, and panel c shows bounded potential aggregation, temporal reward adaptation, and PPO deployment.

3.1 Portfolio state, action, and execution

For each of five risky assets, the raw input contains a 20-trading-day history of close, open, high, low, volume, and adjusted close. The environment extracts ten price descriptors per asset: the five most recent closes relative to the 20-day mean and the five most recent close-to-close returns. It then appends finite features produced by `revise_state`, a five-dimensional optional portfolio slot, a three-component market-regime vector, and the six current allocations. Missing optional features are filled with zeros. If `revise_state` appends K finite features per asset, the policy state has $64 + 5K$ dimensions; generated features augment rather than replace the fixed representation.

The actor maps this state to positive Dirichlet concentration parameters. Training samples a long-only simplex allocation, whereas evaluation uses the Dirichlet mean. Let $\mathbf{w}_t$ denote the current weights including cash, $\mathbf{a}_t$ the target allocation, $\mathbf{u}_{t+1}$ the next return vector with zero cash return, d_t the drawdown from the running wealth peak, c the turnover cost, and λ the drawdown coefficient. The environment credits the price transition to the existing holdings and then applies the new target:

$$\begin{aligned} r_t^{\text{net}} &= \mathbf{w}_t^\top \mathbf{u}_{t+1} - c \frac{\|\mathbf{a}_t - \mathbf{w}_t\|_1}{2}, \\ r_t^{\text{base}} &= r_t^{\text{net}} - \lambda d_t^2, \quad \mathbf{w}_{t+1} = \mathbf{a}_t. \end{aligned}$$

The action at time t therefore becomes the holding for the following transition and does not retrospectively earn the current return. The selected λ is frozen with the code artifact. A separate rule term can reward or penalize concentration, diversification, turnover, defensive cash, momentum alignment, volatility scaling, and drawdown behavior.

3.2 Generated state–reward interface

Following GIFT [33], the LLM produces two Python callables. `revise_state` must preserve the 120 raw entries and may append finite features; `intrinsic_reward` must return a finite scalar. Candidate programs pass abstract-syntax filtering and restricted-namespaces checks for function presence, prefix preservation, output dimensions, finiteness, and scalar range. These safeguards validate the expected interface but do not constitute operating-system isolation or prove that arbitrary generated code is safe.

Within each GIFT iteration, a valid interface and its rule configuration remain fixed while a newly initialized PPO agent is trained. Training-return Sharpe selects candidates, and information-coefficient and critic-attribution diagnostics provide textual feedback for later refinements [33]. The selected callables, rule parameters, and drawdown coefficient are then frozen. Transfer refers to this interface rather than policy weights: every downstream period trains a fresh PPO agent. Pure GIFT adds the bounded scalar at the next market state directly to the financial and rule rewards.

3.3 Bounded potential reward and PPO deployment

PBIR-GIFT retains the name `intrinsic_reward` so the sandbox and controller APIs remain unchanged, but changes its mathematical role. For each asset, the function receives the revised 20-day history and returns a state score. The environment keeps finite values, averages them over the valid asset set, and clips the result to the configured bound. It then forms the shaping reward from consecutive potentials:

$$\begin{aligned} \Phi(s_t) &= \text{clip}\left(\frac{1}{|\mathcal{V}_t|} \sum_{i \in \mathcal{V}_t} \phi_i(s_t), -b, b\right), \\ F_t &= \eta [\gamma \Phi(s_{t+1}) - \Phi(s_t)], \\ r_t^{\text{PBIR}} &= r_t^{\text{base}} + r_t^{\text{rule}} + F_t. \end{aligned}$$

If no valid score is available or either callable fails, the potential falls back to zero. Aggregation and clipping prevent one asset from setting the portfolio-level signal directly. The prompt asks for market-state desirability, such as favorable momentum with lower downside risk and volatility, and excludes realized portfolio return. The potential therefore depends on market history and generated market features, not on current allocations, wealth, or the selected action.

The environment reads γ from the PPO configuration and rejects a mismatch. At a terminal transition, the next potential is zero; no nonlinear clipping is applied after the difference. For a fixed bounded deterministic potential, matching discount factors, a fixed initial-state distribution, and zero terminal potential, the discounted shaping sum is

$$\sum_{t=0}^{T-1} \gamma^t F_t = \eta [-\Phi(s_0) + \gamma^T \Phi(s_T)] = -\eta \Phi(s_0).$$

This policy-independent constant preserves the optimal policies of the reference financial-plus-rule reward under the stated assumptions [21]. It does not preserve the objective induced by Pure GIFT’s direct scalar, guarantee identical finite-sample PPO iterates, ensure faster convergence or superior returns, or eliminate reward hacking under function approximation. On a fixed historical price path, differences between the variants reflect the sensitivity of value estimation and policy optimization to reward timing rather than an ability to steer the market into a higher-potential state.

Optimization and deployment. PPO uses a Dirichlet actor and two value critics. Each update follows one complete historical episode. Generalized advantage estimation [25] uses the lower critic estimate; advantages are standardized, the actor follows PPO’s clipped objective [26] with entropy regularization, and the critics use clipped mean-squared error. Generated callables, rule parameters, and the PBIR switch remain fixed within each training call. A later evaluation period reloads the selected interface, trains a fresh PPO policy on its adaptation segment, and uses deterministic mean allocations without online LLM calls. The language model is therefore a design-time component rather than a decision-time trading oracle.

4 Experiments

4.1 Experimental settings

The following subsections define the datasets, comparison protocols, parameter search, and evaluation criteria. All settings were fixed before the final evaluation; no outcome is used to redefine the protocol.

4.1.1 Dataset source

We use two five-equity datasets derived from the FINSABER S&P 500 release [33]. **Dataset 1** and **Dataset 2** each contain 3,524 trading dates from 29 June 2010 to 28 June 2024 and six market channels per equity. Both datasets are evaluated over six rolling half-year windows, each preceded by an adaptation period and a 20-day state warm-up. Windows 1-5 contain 105 evaluated daily returns and Window 6 contains

104. The datasets share three equities and therefore provide complementary robustness evidence rather than independent asset universes.

4.1.2 Comparison algorithms

We evaluate **ten comparison methods in three roles**:

- **Core mechanism control:** Pure GIFT uses the same cached code, rule configuration, artifact hash, and PPO seed as PBIR-GIFT; only the reward transformation changes.
- **Reinforcement-learning controls:** matched-budget PPO and independently tuned PPO [26] test sensitivity to the RL baseline.
- **External strategy controls:** Equal Weight, SMA, WMA, ATR, Bollinger Bands, Turn-of-the-Month, and XGBoost cover rule-based and supervised alternatives [2, 5, 6].

The cached comparison estimates the PBIR effect conditional on a selected artifact. External comparisons use recorded full-pipeline GIFT artifacts and are reported separately; no transitive comparison is made across protocols.

4.1.3 Parameter adjustment

Stage A searches **five parameter groups in a fixed order** on Dataset 1, Window 1:

- PPO update epochs: 1, 3, 5, 8, and 10;
- maximum training episodes: 20, 35, 50, 75, and 100;
- actor learning rate: 0.00005, 0.0001, 0.0003, 0.0005, and 0.001;
- PPO clipping coefficient: 0.05, 0.10, 0.15, 0.20, and 0.30;
- entropy coefficient: 0, 0.001, 0.005, 0.01, and 0.02.

The January-June 2020 interval is divided into adaptation (2 January-31 March) and validation (1 April-30 June). Each step maximizes the mean validation Sharpe across the two methods and freezes earlier selections. The 25 paired selection trials use seed 42; final cached evaluation uses seeds 42, 123, and 456, while PPO is tuned independently with the same three seeds. Because deep-RL results are seed-sensitive [1, 12] and repeated search can inflate backtest performance [3, 31], these curves are configuration diagnostics rather than causal hyperparameter evidence.

4.1.4 Evaluation metrics

Evaluation uses **five performance measures and four inferential checks**:

- **Primary measure:** annualized Sharpe with a zero risk-free rate [27].
- **Secondary measures:** Sortino [28], compounded period return, maximum draw-down (MDD), and Calmar. Higher values are preferable except for MDD.
- **Analysis unit:** one dataset-window cell, averaged over three seeds before paired comparison.
- **Inference:** positive-cell counts, Hedges' effect size, an exact one-sided sign-flip test, and 10,000 hierarchical circular block-bootstrap resamples [24]. Holm correction is applied within each declared comparison family [13].

- **Strict PBIR criterion:** a positive lower Sharpe confidence bound, positive mean differences in both datasets, at least 8 of 12 positive cells, and no MDD deterioration above five percentage points.

4.2 Experimental results

The main text reports the shortest evidence chain for accuracy, cross-dataset generalization, efficiency, and comparison boundaries. Hyperparameter-selection curves and input-output diagnostics are retained in the Appendix.

4.2.1 Accuracy and cross-dataset generalization

PBIR-GIFT increased mean cell-level Sharpe by 0.278 over Pure GIFT across 12 fixed-code dataset-window cells. Ten cells were positive (Hedges' $g = 0.940$; one-sided sign-flip $p = 0.0044$; five-metric Holm-adjusted $p = 0.022$), but the hierarchical-bootstrap 95% interval $[-0.010, 0.558]$ crossed zero. The prespecified strict superiority gate therefore failed, supporting a favorable direction rather than universal superiority.

The mean Sharpe difference was positive in Dataset 1 (+0.220; five of six windows) and Dataset 2 (+0.336; five of six), and in odd (+0.368) and even (+0.188) windows. Window-averaged differences ranged from -0.027 in Window 4 to +0.545 in Window 2. This pattern was not confined to one dataset, although shared assets and temporally related windows limit independence.

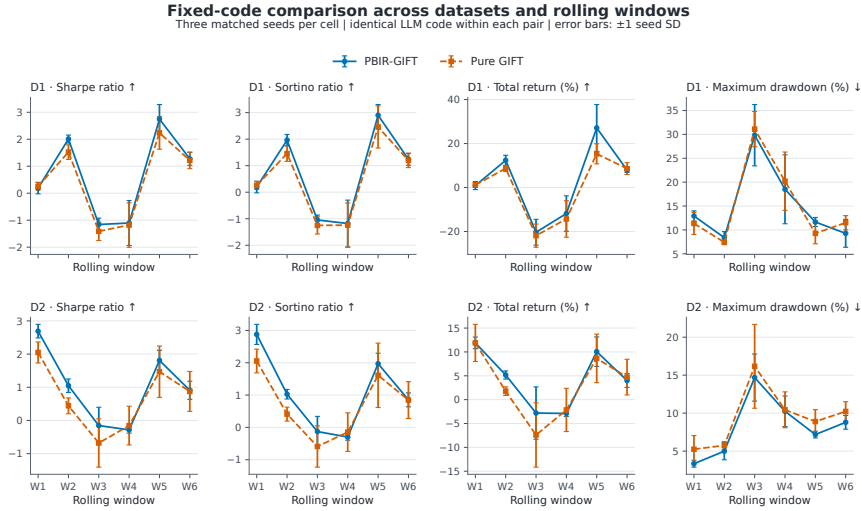


Fig. 2 Cross-dataset and cross-window evaluation. Points are three-seed means and whiskers are one sample standard deviation. Returns and MDD are percentages; lower MDD is preferable.

Secondary point estimates were also favorable but uncertain: Sortino +0.276, return +2.205 percentage points, MDD improvement +0.647 points, and Calmar +0.675; every 95% interval crossed zero. The worst cell-level MDD deterioration was 2.375 points, within the five-point guardrail.

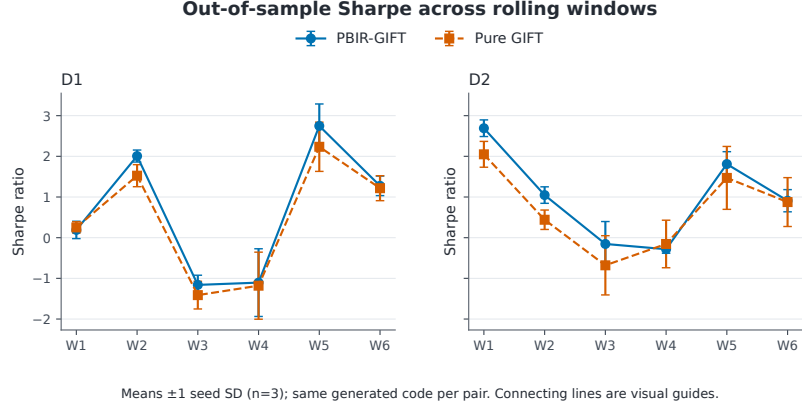


Fig. 3 Fixed-code Sharpe by dataset and window. Means plus or minus one seed standard deviation ($n = 3$); paired methods share identical generated code and differ only in reward transformation.

4.2.2 Efficiency and baseline boundary

All 72 cached method runs made zero LLM calls, isolating downstream training from repeated code generation. The median PBIR-to-Pure runtime ratio was 1.008 across 36 pairs, but unstandardized hardware load and one 9.30-hour outlier preclude speed or runtime-equivalence claims (Fig. 4).

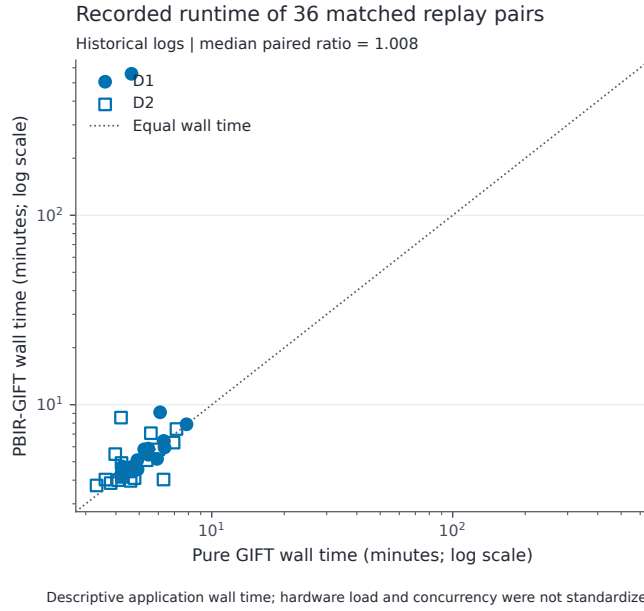


Fig. 4 Recorded application runtimes. Each point is one matched dataset-window-seed pair; the diagonal denotes equal runtime. Timings are descriptive, not a controlled GPU benchmark.

Baseline comparisons further bound the claim (Fig. 5). Pure GIFT differed from matched PPO by +0.112 and from independently tuned PPO by -0.144; neither interval excluded zero. Its positive contrasts with SMA, Bollinger Bands, and XGBoost also failed the joint multiplicity-and-interval gate. Thus, the strongest evidence remains the conditional PBIR-versus-Pure GIFT comparison under shared code, not general GIFT superiority over strong PPO.

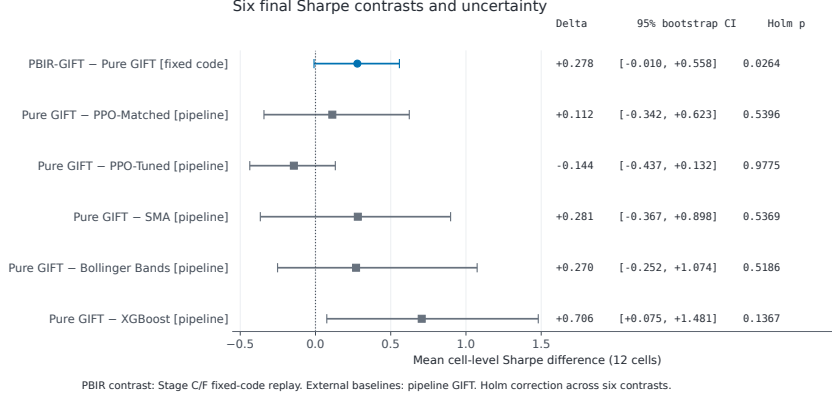


Fig. 5 Six final Sharpe contrasts. Horizontal intervals are 95% hierarchical block-bootstrap intervals; the right column reports Holm-adjusted one-sided sign-flip values. The PBIR row uses fixed-code results, whereas external baselines use their recorded full-pipeline sources, so rows are not transitive comparisons.

5 Conclusion

PBIR-GIFT reinterprets GIFT’s generated scalar as a bounded market-state potential and applies an environment-side temporal-difference reward while retaining the original callable interface. Across two datasets and six windows, Sharpe improved by 0.278 in 10 of 12 fixed-code cells, although the 95% interval marginally crossed zero and strong PPO superiority was not established. Future work should tune potential scale and portfolio-aware potentials on preregistered, non-overlapping markets while benchmarking runtime and transaction-cost sensitivity under standardized hardware.

Appendix A Hyperparameter selection

A.1 Sequential selection results

Stage A selected 10 PPO update epochs, 50 training episodes, an actor learning rate of 0.0005, a PPO clipping coefficient of 0.30, and an entropy coefficient of 0.001. Each level was run once with seed 42 and independently generated code, so the plots document configuration selection rather than causal hyperparameter effects or uncertainty intervals.

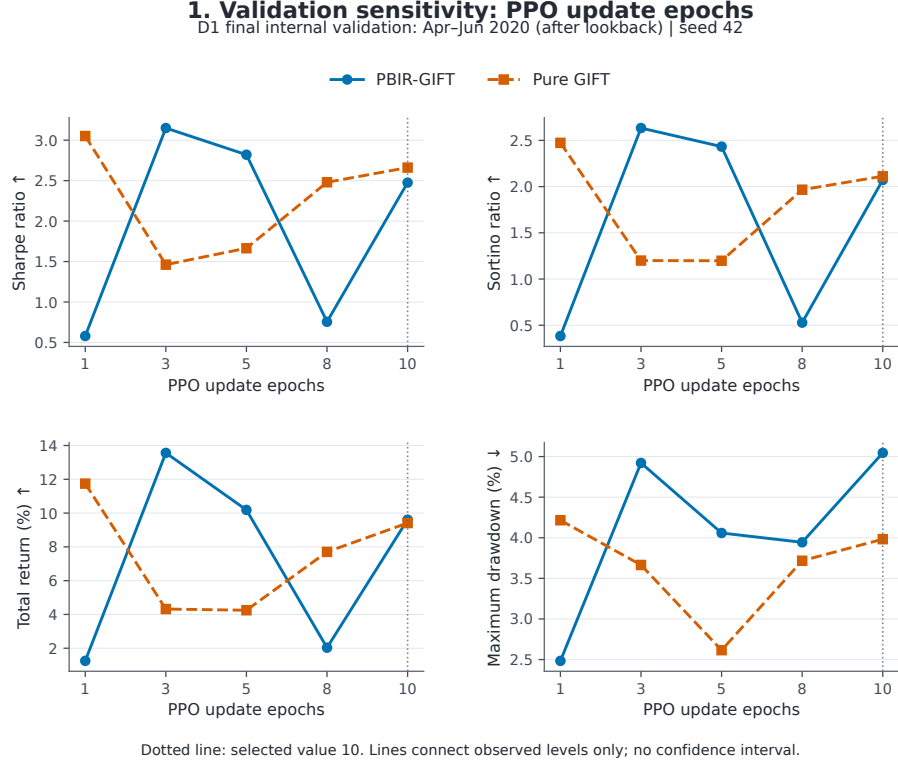


Fig. 6 Update-epoch sensitivity. Internal-validation metrics for PBIR-GIFT and Pure GIFT; the dotted line marks the joint selection.

2. Validation sensitivity: Training episodes

D1 final internal validation: Apr-Jun 2020 (after lookback) | seed 42

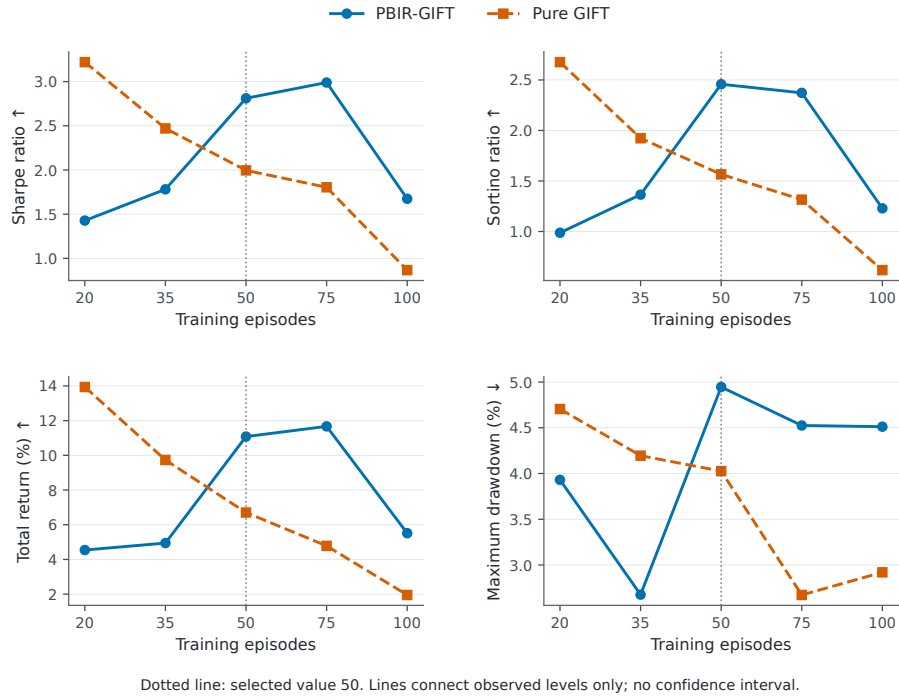
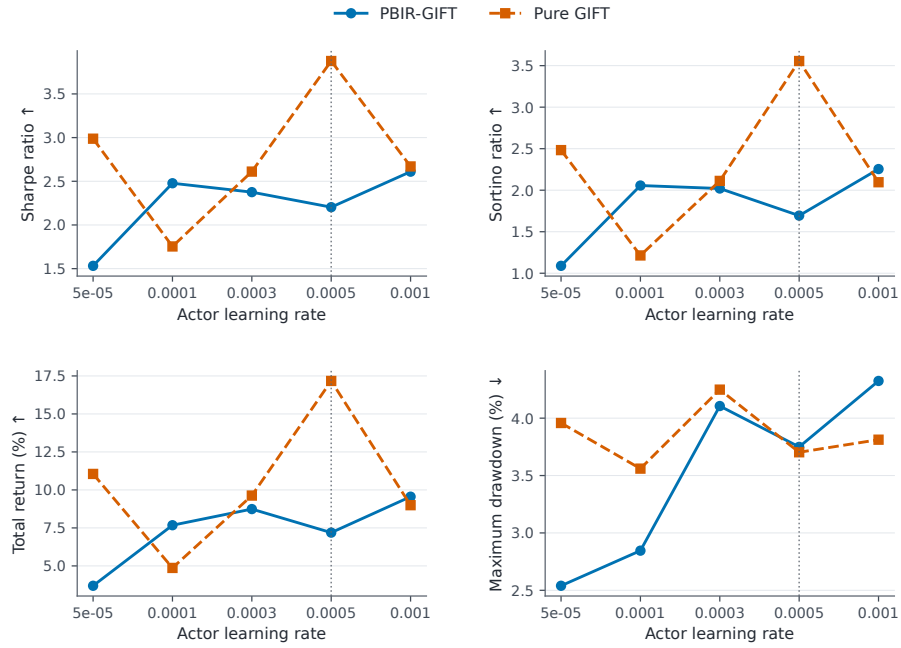


Fig. 7 Training-episode sensitivity. Internal-validation metrics after freezing the epoch choice; the dotted line marks the joint selection.

3. Validation sensitivity: Actor learning rate

D1 final internal validation: Apr-Jun 2020 (after lookback) | seed 42



Dotted line: selected value 0.0005. Lines connect observed levels only; no confidence interval.

Fig. 8 Actor-learning-rate sensitivity. Internal-validation metrics after freezing prior selections; the dotted line marks the selected rate.

4. Validation sensitivity: PPO clipping coefficient

D1 final internal validation: Apr-Jun 2020 (after lookback) | seed 42

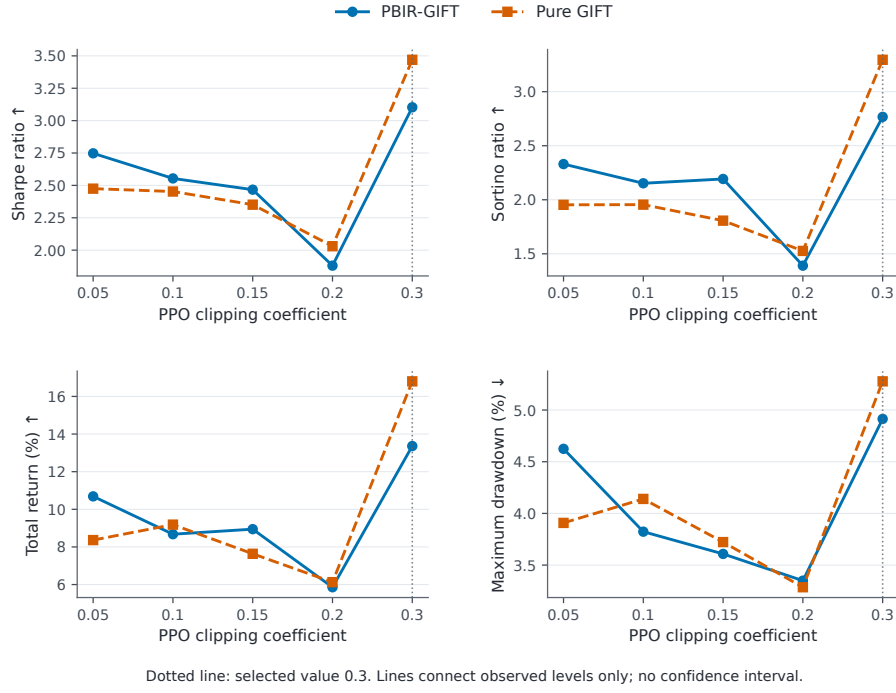


Fig. 9 PPO-clipping sensitivity. The selected coefficient lies at the search boundary and is therefore the best observed level, not an unconstrained optimum.

5. Validation sensitivity: Entropy coefficient

D1 final internal validation: Apr-Jun 2020 (after lookback) | seed 42

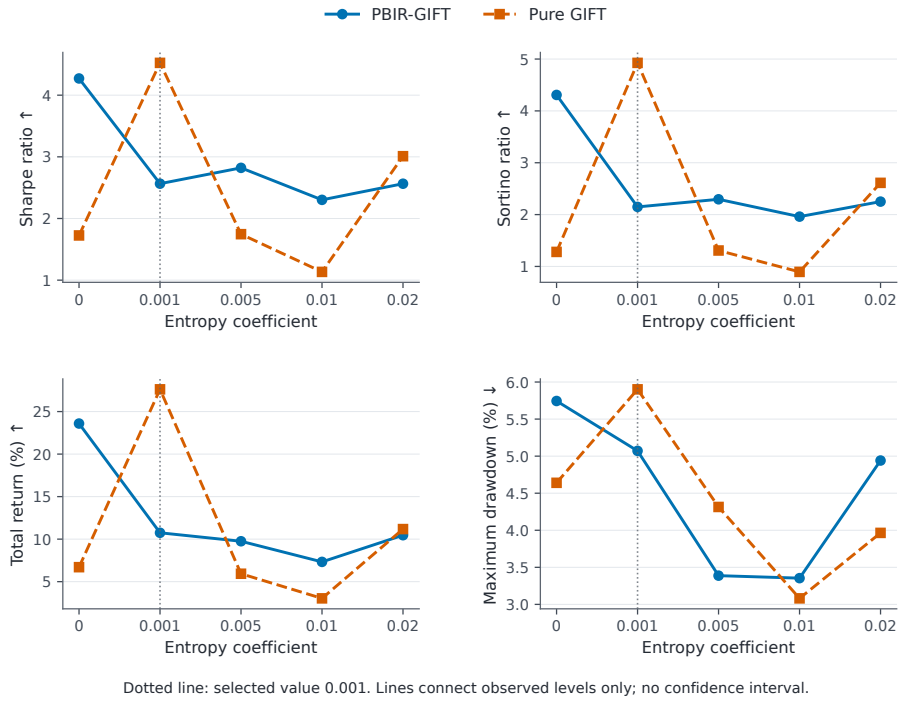


Fig. 10 Entropy-coefficient sensitivity. Internal-validation metrics after freezing the first four parameter groups; the dotted line marks the joint selection.

Appendix B Input and output diagnostics

The diagnostic figures report all input trajectories, wealth paths, and average allocations rather than selected successful cases. They support traceability and behavioral inspection but do not identify a causal allocation mechanism.

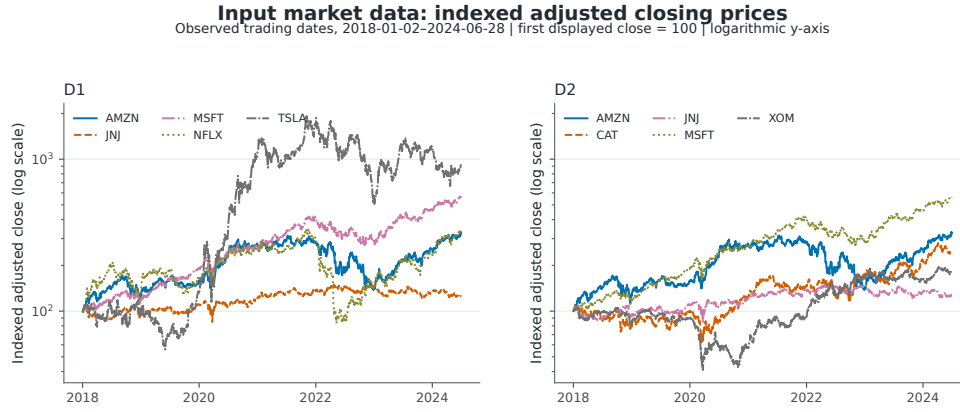


Fig. 11 Input data in Dataset 1 and Dataset 2. Adjusted closes are normalized to 100 on 2 January 2018 and shown on a log scale. No values are imputed, and normalization is not the model's preprocessing transform.

Output portfolio wealth in each evaluation window

Initial wealth = 100 per window | mean of three seed-specific wealth paths | no stitching

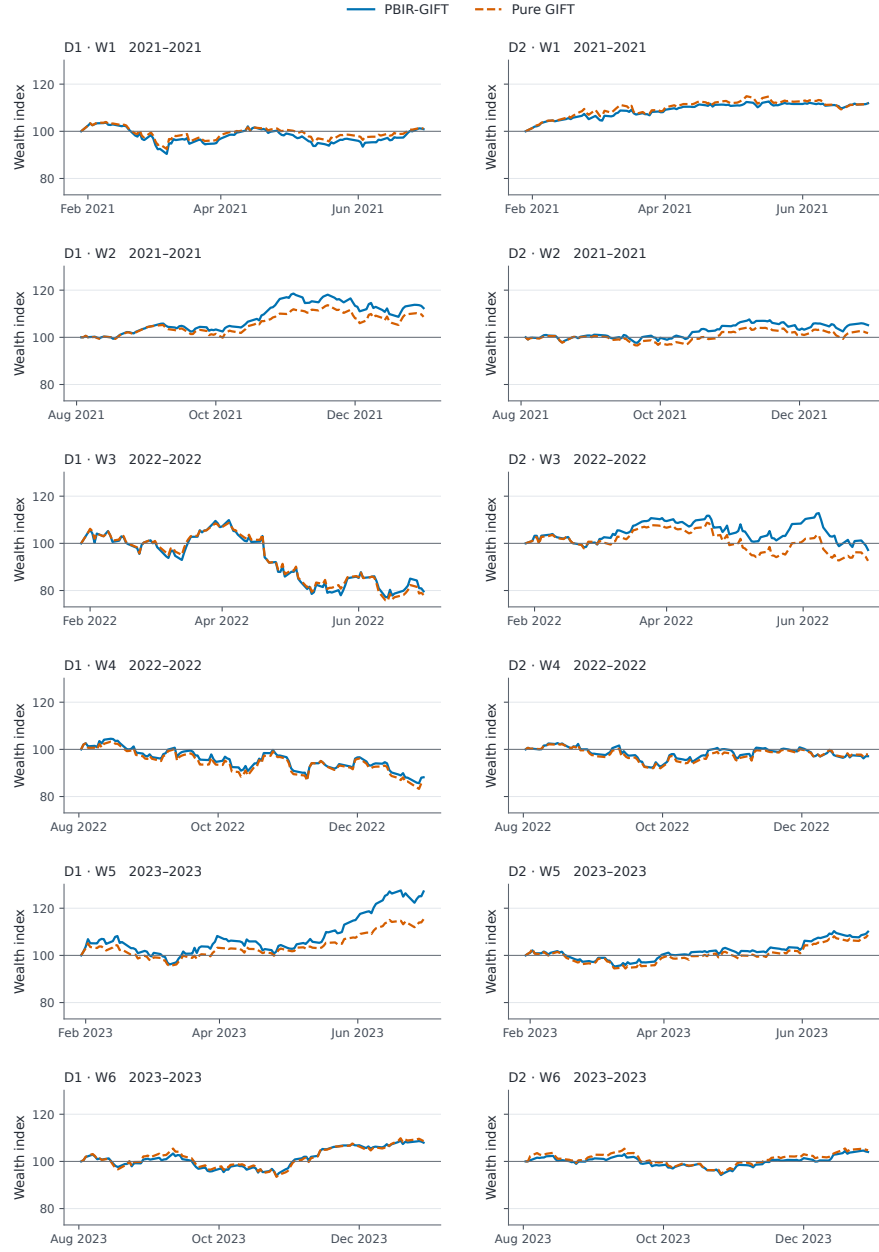


Fig. 12 Output wealth paths for all 12 dataset-window cells. Each path starts at 100 and averages three seed-specific wealth indices. Windows restart independently; no smoothing or confidence ribbon is applied.

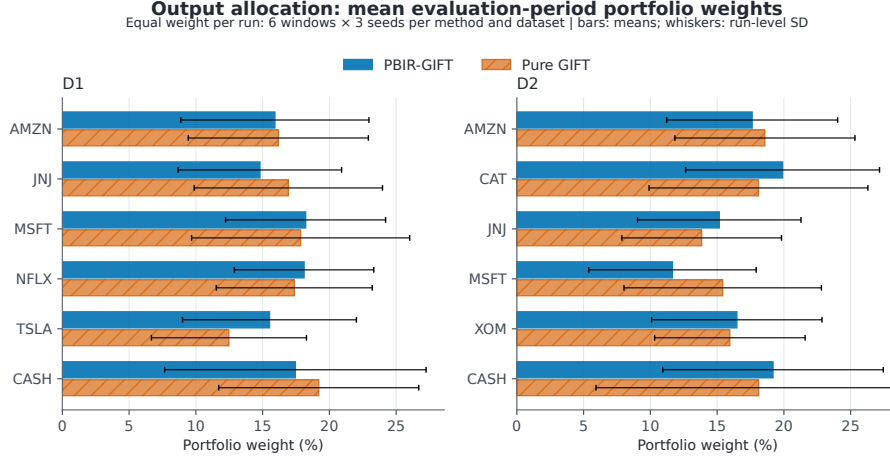


Fig. 13 Evaluation-period allocation outputs. Bars average all 18 window-seed run weights per method and dataset, including cash; whiskers are descriptive standard deviations, not confidence intervals.

Declarations

Funding *Author input required before submission.*

Competing interests *Author input required before submission.*

Author contributions *Author names and contribution roles are required before submission.*

Data availability The experiments use two processed five-equity subsets derived from the FINSABER benchmark cited in this article. The public repository will provide preprocessing instructions and integrity checks; redistribution of processed market data remains subject to the source dataset’s terms.

Code availability The implementation, experiment manifests, and reproducibility instructions are prepared for release in the accompanying GitHub repository. The final public URL and archived release identifier must be inserted before submission.

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

References

- [1] Agarwal R, Schwarzer M, Castro PS, et al (2021) Deep reinforcement learning at the edge of the statistical precipice. In: Advances in Neural Information Processing Systems, URL <https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html>
- [2] Ariel RA (1987) A monthly effect in stock returns. Journal of Financial Economics 18(1):161–174. [https://doi.org/10.1016/0304-405X\(87\)90066-3](https://doi.org/10.1016/0304-405X(87)90066-3), URL [https://doi.org/10.1016/0304-405X\(87\)90066-3](https://doi.org/10.1016/0304-405X(87)90066-3)

- [3] Bailey DH, López de Prado M (2014) The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management* 40(5):94–107. <https://doi.org/10.3905/jpm.2014.40.5.094>, URL <https://doi.org/10.3905/jpm.2014.40.5.094>
- [4] Benhenda M (2025) FinRL-DeepSeek: LLM-infused risk-sensitive reinforcement learning for trading agents. URL <https://arxiv.org/abs/2502.07393>, [arXiv:2502.07393](https://arxiv.org/abs/2502.07393)
- [5] Brock W, Lakonishok J, LeBaron B (1992) Simple technical trading rules and the stochastic properties of stock returns. *The Journal of Finance* 47(5):1731–1764. <https://doi.org/10.1111/j.1540-6261.1992.tb04681.x>, URL <https://doi.org/10.1111/j.1540-6261.1992.tb04681.x>
- [6] Chen T, Guestrin C (2016) XGBoost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp 785–794, <https://doi.org/10.1145/2939672.2939785>, URL <https://doi.org/10.1145/2939672.2939785>
- [7] Chen Y, Yao Z, Liu Y, et al (2025) StockBench: Can LLM agents trade stocks profitably in real-world markets? URL <https://arxiv.org/abs/2510.02209>, [arXiv:2510.02209](https://arxiv.org/abs/2510.02209)
- [8] Darmanin A, Vella V (2025) Language model guided reinforcement learning in quantitative trading. In: *Proceedings of FLLM*, pp 405–412
- [9] Goluža S, Kovačević T, Bauman T, et al (2024) Deep reinforcement learning with positional context for intraday trading. *Evolving Systems* 15(5):1865–1880
- [10] Goluža S, Kovačević T, Begušić S, et al (2024) Robot see, robot do: Imitation reward for noisy financial environments. In: *Proceedings of the IEEE International Conference on Big Data*, pp 4884–4891
- [11] Hambly B, Xu R, Yang H (2023) Recent advances in reinforcement learning in finance. *Mathematical Finance* 33(3):437–503. <https://doi.org/10.1111/mafi.12382>, URL <https://doi.org/10.1111/mafi.12382>
- [12] Henderson P, Islam R, Bachman P, et al (2018) Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence* 32(1):3207–3214. <https://doi.org/10.1609/aaai.v32i1.11694>, URL <https://doi.org/10.1609/aaai.v32i1.11694>
- [13] Holm S (1979) A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2):65–70. <https://doi.org/10.2307/4615733>, URL <https://doi.org/10.2307/4615733>

- [14] Jiang Z, Xu D, Liang J (2017) A deep reinforcement learning framework for the financial portfolio management problem. URL <https://arxiv.org/abs/1706.10059>, [arXiv:1706.10059](#)
- [15] Karzanov D, Garzón R, Terekhov M, et al (2025) Regret-optimized portfolio enhancement through deep reinforcement learning and future-looking rewards. In: Proceedings of the ACM International Conference on AI in Finance, pp 890–897
- [16] Lee J, Kim R, Yi SW, et al (2020) MAPS: Multi-agent reinforcement learning-based portfolio management system. In: Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, pp 4520–4526
- [17] Liang Z, Chen H, Zhu J, et al (2018) Adversarial deep reinforcement learning in portfolio management. URL <https://arxiv.org/abs/1808.09940>, [arXiv:1808.09940](#)
- [18] Lien YH, Li YK, Wang YS (2023) Contrastive learning and reward smoothing for deep portfolio management. In: Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, pp 3966–3974
- [19] Liu XY, Xia Z, Rui J, et al (2022) FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning. In: Advances in Neural Information Processing Systems, <https://doi.org/10.52202/068431-0134>, URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/0bf54b80686d2c4dc0808c2e98d430f7-Abstract.html
- [20] Moody J, Saffell M (2001) Learning to trade via direct reinforcement. IEEE Transactions on Neural Networks 12(4):875–889
- [21] Ng AY, Harada D, Russell S (1999) Policy invariance under reward transformations: Theory and application to reward shaping. In: Proceedings of the Sixteenth International Conference on Machine Learning, pp 278–287
- [22] Nie Y, Kong Y, Dong X, et al (2024) A survey of large language models for financial applications: Progress, prospects and challenges. URL <https://arxiv.org/abs/2406.11903>, [arXiv:2406.11903](#)
- [23] Pippas N, Turkay C, Ludvig EA (2025) The evolution of reinforcement learning in quantitative finance: A survey. ACM Computing Surveys 57(11):1–51. <https://doi.org/10.1145/3733714>, URL <https://doi.org/10.1145/3733714>
- [24] Politis DN, Romano JP (1992) A circular block-resampling procedure for stationary data. In: LePage R, Billard L (eds) Exploring the Limits of Bootstrap. Wiley, New York, p 263–270
- [25] Schulman J, Moritz P, Levine S, et al (2015) High-dimensional continuous control using generalized advantage estimation. URL <https://arxiv.org/abs/1506.02438>, [arXiv:1506.02438](#)

- [26] Schulman J, Wolski F, Dhariwal P, et al (2017) Proximal policy optimization algorithms. URL <https://arxiv.org/abs/1707.06347>, arXiv:1707.06347
- [27] Sharpe WF (1966) Mutual fund performance. The Journal of Business 39(S1):119–138. <https://doi.org/10.1086/294846>, URL <https://doi.org/10.1086/294846>
- [28] Sortino FA, van der Meer R (1991) Downside risk: Capturing what’s at stake in investment situations. The Journal of Portfolio Management 17(4):27–31. <https://doi.org/10.3905/jpm.1991.409343>, URL <https://doi.org/10.3905/jpm.1991.409343>
- [29] Sun S, Wang R, An B (2023) Reinforcement learning for quantitative trading. ACM Transactions on Intelligent Systems and Technology 14(3):44:1–44:29. <https://doi.org/10.1145/3582560>, URL <https://doi.org/10.1145/3582560>
- [30] Wang Z, Huang B, Tu S, et al (2021) DeepTrader: A deep reinforcement learning approach for risk–return balanced portfolio management with market conditions embedding. Proceedings of the AAAI Conference on Artificial Intelligence 35(1):643–650. <https://doi.org/10.1609/aaai.v35i1.16144>, URL <https://doi.org/10.1609/aaai.v35i1.16144>
- [31] White H (2000) A reality check for data snooping. Econometrica 68(5):1097–1126. <https://doi.org/10.1111/1468-0262.00152>, URL <https://doi.org/10.1111/1468-0262.00152>
- [32] Wu S, Irsoy O, Lu S, et al (2023) BloombergGPT: A large language model for finance. URL <https://arxiv.org/abs/2303.17564>, arXiv:2303.17564
- [33] Wu Y, Zhang B, Liu Y, et al (2026) GIFT: LLM-guided state–reward interface for financial reinforcement learning. URL <https://arxiv.org/abs/2606.08450>, arXiv:2606.08450
- [34] Xie Q, Han W, Chen Z, et al (2024) FinBen: A holistic financial benchmark for large language models. In: Advances in Neural Information Processing Systems
- [35] Xiong G, Deng Z, Wang K, et al (2025) FLAG-Trader: Fusion LLM-agent with gradient-based reinforcement learning for financial trading. URL <https://arxiv.org/abs/2502.11433>, arXiv:2502.11433
- [36] Xu K, Zhang Y, Ye D, et al (2020) Relation-aware transformer for portfolio policy learning. In: Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, pp 4647–4653, <https://doi.org/10.24963/ijcai.2020/641>, URL <https://doi.org/10.24963/ijcai.2020/641>
- [37] Yang H, Liu XY, Wang CD (2023) FinGPT: Open-source financial large language models. URL <https://arxiv.org/abs/2306.06031>, arXiv:2306.06031

- [38] Yang M, Zheng X, Liang Q, et al (2022) A smart trader for portfolio management based on normalizing flows. In: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, pp 4014–4021, <https://doi.org/10.24963/ijcai.2022/557>, URL <https://doi.org/10.24963/ijcai.2022/557>
- [39] Yu Y, Yao Z, Li H, et al (2024) FinCon: A synthesized LLM multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. In: Advances in Neural Information Processing Systems
- [40] Zhang K, Yoshie O, Sun L, et al (2025) FinLLM-B: When large language models meet financial breakout trading. In: Proceedings of the NAACL Industry Track, pp 349–357
- [41] Zhao L, Kong S, Shen Y (2023) DoubleAdapt: A meta-learning approach to incremental learning for stock trend forecasting. In: Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp 3492–3503
- [42] Zong C, Wang C, Qin M, et al (2024) MacroHFT: Memory augmented context-aware reinforcement learning on high-frequency trading. In: Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp 4712–4721